

# Laboratório 05

## Parte 3

Neste laboratório o objetivo é implementar um microcontrolador simples de 16 bits. O laboratório está dividido em 5 partes, e neste documento iremos detalhar apenas a terceira destas.

### Parte 3

Nesta parte você deve implementar o circuito da Figura 7. Observe que as principais mudanças em relação ao processador projetado na Parte 1 se referem ao registrador R7 (que, além de registrador, também atuará como o *Program Counter (PC)* e, portanto, possui uma entrada adicional (*incr\_pc*) que incrementa em 1 o valor armazenado, além de sinais adicionais (*ADDRin*, *DOUTin*, *Wd*) de saída da máquina de estados que estarão conectados aos registradores da memória (que estão ilustrados na Figura 7 e também devem ser implementados, mesmo sabendo que o componente da memória também implementa tais registradores automaticamente).

Três novas instruções devem ser implementadas, conforme apresentado na Tabela 3:

- A instrução **ld** Rx,Ry (com opcode 100) coloca em Rx o conteúdo da memória cujo endereço se encontra em Ry.
- A instrução **st** Rx,Ry (com opcode 101) coloca na memória cujo endereço se encontra em Rx o conteúdo de Ry.
- Finalmente, a instrução **mvnz** Rx,Ry (opcode 110) realiza a instrução de cópia do conteúdo de Ry para Rx caso o valor de G não seja zero.

A implementação da última instrução pode ser facilitada criando-se um novo sinal de entrada da máquina de estados, vindo da saída de um comparador de G com o valor 0. Observe que, com esta construção, podemos realizar saltos no programa (JMP – equivalente a **mv** para o registrador R7) e saltos condicionais (JNZ – equivalente a **mvnz** para R7).

A conexão da memória com o processador é apresentada na Figura 8. Alguns pontos são importantes destacar. Primeiramente, como na parte 2, o conteúdo da memória entra diretamente no DIN do processador. Desta forma, a memória deve ser inicializada com os dados do programa. Porém, ela também poderá ser escrita, de forma que deverá ser, portanto, do tipo RAM. Além disso, diferentemente da parte 2, não temos um contador externo (e um clock específico) para atualizar, de forma que o próprio processador deverá ser modificado de forma a buscar na memória a instrução que deverá executar (conhecido por *fetch*).

Especificamente, enquanto que no processador da parte 1, o primeiro ciclo (T0), comum a todas as instruções, copiava parte de DIN para IR, agora todas as instruções devem implementar o *fetch* antes do T0. Assim, devemos ter novos ciclos (TA e TB), que antecedem T0 em todas as instruções. TA faz com que o barramento contenha R7 (*R7out*), que ative o incremento do Counter (*incr\_pc*) e que ative o registrador de endereços da memória (*ADDRin*). Em TB aguardamos o sincronismo das memórias (relembre dos dois níveis de registradores, tanto da saída do processador quanto de entrada da memória). Assim, sabemos que durante o próximo

clock, DIN terá o conteúdo de memória apontada pelo PC, este já estará apontando para o próximo endereço, e podemos passar para T0.

Note que a instrução **mvi** deverá ser modificada para incluir um novo *fetch* de memória, com outro incremento de R7. Existe uma forma de **mvi** ainda terminar em T1 (ou seja, segue os estados TA->TB->T0->T1), assim como o **mv**. Se descobrir, implemente e indique no relatório como foi feito. Verifique sobretudo que o caso **mvi R7,#D** esteja funcionando corretamente, sobretudo nas questões de prioridade entre o incremento e a cópia neste caso em particular.

Observe na Figura 8 a existência de 2 componentes NOR de 4 bits. Observe que o papel dele é verificar se os 4 bits mais significativos de A são 0000 para habilitar a escrita na memória e 0001 para habilitar a escrita no registrador dos LEDs. Desta forma, estamos definindo faixas de endereços tanto para a memória quanto para os periféricos. Assim, a mesma instrução **st** permite gravar um conteúdo na memória (caso o endereço comece por 0000) quanto em um periférico (no caso o LED, caso o endereço comece por 0001), de forma que esta diferença é transparente para o processador.

Usaremos como código para teste o seguinte arquivo .mif (que deverá ser convertido em .hex para que possa ser simulado).

```
DEPTH = 128;
WIDTH = 16;
ADDRESS_RADIX = HEX;
DATA_RADIX = BIN;

CONTENT BEGIN

% Este código apresenta um contador (no registrador R2) nos LEDs vermelhos. %
00 : 0010010000000000;      %      mvi    R1,#1 %
01 : 0000000000000001;
02 : 0010100000000000;      %      mvi    R2,#0 [LED] %
03 : 0000000000000000;
04 : 0010110000000000;      %      mvi    R3,#0001000000000000 end. do reg. do LED %
05 : 0001000000000000;
06 : 1010100110000000;      %      Loop: st R2,R3 escreve R2 nos LEDs %
07 : 0100100010000000;      %      add    R2,R1  ++[LED] %
08 : 0011000000000000;      %      mvi    R4,#0000000000000011 Atraso %
09 : 0000000000000011;
0A : 0001011110000000;      %      mv     R5,R7  Salva o end. da próxima instrução %
0B : 0111000010000000;      %      sub    R4,R1  Decrementa R4 %
0C : 1101111010000000;      %      mvnz   R7,R5  jnz Atraso %
0D : 0011110000000000;      %      mvi    R7,#Loop %
0E : 0000000000000110;

END;
```

Observe que no endereço 06 criamos um loop principal que armazena no endereço mapeado pelos LEDR valores crescentes. Em 0A-0C criamos um loop de atraso, e em 0D-0E efetuamos um JMP de volta para Loop.

A entidade a ser usada é:

```
ENTITY lab12part2 IS
    PORT ( Resetn, Clock, Run      : IN  STD_LOGIC;
          LEDR                      : OUT  STD_LOGIC_VECTOR(15 DOWNTO 0);
          DONE                      : OUT  STD_LOGIC);
END lab12part2;
```

O resultado da simulação obtida foi:

